IronMind AI — Technical Project Manual & Guide Briefing

Real-Time Edge-AI Biomechanical Posture Coach & Kinematic Movement Analysis Platform

Architecture: Flutter (Dart 3.x) • Google ML Kit (BlazePose) • FastAPI (Python 3.12) • SQLite 3 • Standalone Python Prototype

1. Executive Summary & Problem Statement

IronMind AI addresses the critical issue of improper exercise form during unassisted workouts, which is the primary cause of musculoskeletal injuries and sub-optimal athletic progression. Conventional solutions require either expensive wearable sensory equipment or cloud-streamed video analysis that suffers from severe network latency (100–300ms) and compromises user visual privacy.

IronMind AI delivers an **on-device, sub-30ms real-time computer vision platform** that tracks 33 3D anatomical landmarks via Google ML Kit (BlazePose), computes trigonometric 3D joint angles using arctangent formulation, filters camera optical sensor jitter via Exponential Moving Average (EMA), and evaluates exercise phases through multi-state finite kinematic state machines.

2. Comprehensive Technology Stack

Layer	Technology	Role & Functional Details
Mobile Frontend	Flutter (Dart 3.x)	Cross-platform 60 FPS UI, Material 3 dark aesthetic, CameraX image stream processing, local state management.
Edge Computer Vision	Google ML Kit (BlazePose)	On-device 33 3D landmark regression neural network running on GPU/NPU delegates with zero cloud latency.
Kinematics & Math	Custom Biomechanical Engine	Trigonometric atan2 joint angle calculation, Exponential Moving Average (EMA) noise filtration, dynamic auto-calibration.
Backend API	FastAPI + Uvicorn (Python 3.12)	High-performance asynchronous REST API for user authentication, workout telemetry sync, and statistical aggregation.
Database & Storage	SQLite 3 + SharedPreferences	**Hybrid Offline-First Design:** Local persistence via Android SharedPreferences + centralized indexed SQLite server.
Security	PBKDF2-HMAC-SHA256	100,000 hashing iterations with 16-byte cryptographically secure random salt and Bearer session tokens.
Secondary Prototype	Pure Python (OpenCV + MediaPipe)	Standalone desktop vision testbed and Android Buildozer packaging configuration.

3. Machine Learning Architecture & Dataset Origin

The system utilizes Google's **BlazePose** 3D pose estimation neural network, structured as a two-stage cascaded pipeline:

- **Stage 1 (Detector Network):** A lightweight convolutional detector (MobileNet backbone) scans the video frame to localize the human body bounding box and key orientation points (hips, shoulders).
- **Stage 2 (Landmark Regression Network):** The cropped region of interest (ROI) is passed to a deep landmark regression model that predicts **33 3D keypoints** (X, Y, Z) normalized between $[0.0, 1.0]$, alongside a visibility score.

Training Datasets:

- **COCO Keypoints Dataset:** Utilized for 2D keypoint pre-training across over 200,000 in-the-wild annotated human images under diverse lighting, clothing, angles, and background variations.
- **Google GHUM 3D Dataset:** A high-fidelity generative 3D human shape and kinematic dataset that enables the model to accurately estimate spatial depth (Z-axis) and resolve self-occlusion (e.g., limb crossing).

4. Biomechanical & Mathematical Formulations

A. 3D Joint Angle Calculation (atan2)

For any 3 consecutive joint landmarks A (proximal), B (vertex), and C (distal):

$$\theta = \left| \text{atan2}(C_y - B_y, C_x - B_x) - \text{atan2}(A_y - B_y, A_x - B_x) \right| \times (180^\circ / \pi)$$
$$\text{if } \theta > 180^\circ \rightarrow \theta = 360^\circ - \theta$$

B. Temporal Smoothing via Exponential Moving Average (EMA)

To eliminate optical noise and sensor jitter from triggering false repetitions:

$$S_t = \alpha \times X_t + (1 - \alpha) \times S_{t-1}$$
$$\text{where } \alpha = 0.18 \text{ for squats / lunges and } \alpha = 0.25 \text{ for push-ups.}$$

C. Dynamic Baseline Auto-Calibration

The system samples $N = 30$ standing baseline frames before active reps to determine individual limb lengths, standing hip height, and shoulder width, dynamically customizing descent, depth, and return thresholds per user.

5. Supported Exercise Kinematics

Exercise	Primary Joints Tracked	Form Validation Rules	State Machine Phases
Squats	Hip → Knee → Ankle	Knee angle < 100° (depth), Back angle > 45°, Knees tracking over toes	STANDING → DESCENDING → BOTTOM_HOLD → ASCENDING → COMPLETE
Push-ups	Shoulder → Elbow → Wrist	Elbow angle < 90° at bottom, Spine/hip straightness ≈ 180°	UP → DESCENDING → BOTTOM → ASCENDING → COMPLETE
Lunges	Hip → Knee → Ankle	Front knee ≈ 90°, Rear knee near ground, Torso upright	START → LUNGING → DEEP_LUNGE → RETURN
Jumping Jacks	Shoulder Abduction & Ankle Span	Shoulder angle > 140° at peak, Feet span wider than shoulder width	REST → PEAK_JUMP → RETURN → COMPLETE
Plank	Shoulder → Hip → Ankle	Continuous core alignment (160°–180°); timer pauses on back sagging	ALIGNMENT → HOLDING → PAUSED → COMPLETE

6. Backend, Database & Offline-First Architecture

- **SQLite 3 Schema:** Dedicated tables for `users`, `profiles`, and `workouts` with B-Tree indices on `users(email)`, `workouts(user_id)`, and `workouts(completed_at)`.
- **Security:** PBKDF2-HMAC-SHA256 with 100,000 iterations + 16-byte cryptographic salt per user.
- **Offline-First Resilience:** Client immediately writes workout metrics to Android `SharedPreferences`. If network is unavailable, the user can seamlessly exercise and sync upon reconnecting.

7. Guide & Viva Defense Q&A Cheatsheet

Q1: Why did you choose Edge-AI / On-Device ML instead of sending video to a cloud server?

Answer: Cloud video streaming requires 30 FPS high-bandwidth upload, creating 150–300ms latency that ruins real-time audio-visual feedback. Edge computing provides sub-30ms response time, functions 100% offline, and guarantees complete user privacy because raw camera frames never leave the smartphone.

Q2: How does the system prevent camera jitter and shaky hands from counting false reps?

Answer: We implement an Exponential Moving Average (EMA) filter with $\alpha = 0.18$ on calculated joint angles to smooth out optical noise, paired with a 4-phase finite state machine that enforces full range-of-motion completion before incrementing reps.

Q3: How are joint angles calculated in 3D space?

Answer: Using the 2-argument arctangent function `atan2(Δy, Δx)` between three consecutive anatomical landmarks (e.g., Hip, Knee, Ankle), computing the interior vector angle normalized between 0° and 180°.

Q4: What happens if the user's phone disconnects from the Wi-Fi or backend?

Answer: The app utilizes an offline-first architecture. It caches athlete profiles and workout sessions locally in Android `SharedPreferences` and synchronizes with the SQLite database via REST API once network connectivity is restored.

Q5: How is user data and password security handled?

Answer: Passwords are never stored in plaintext. They are salted with a 16-byte cryptographically secure random salt and hashed using PBKDF2-HMAC-SHA256 with 100,000 iterations before storage in SQLite.
